Product-company data interview preparation — start here

Who this programme is for

A fresher learning independently for entry-level Data Analyst, Product Analyst, Business Intelligence Analyst or Junior Data Engineer roles. It assumes basic computer literacy but does **not** assume interview-level SQL, DBMS, Python or analytics knowledge.

This is a course, workbook and assessment system. It is deliberately organised as a learning sequence: concept → worked example → guided exercise → independent exercise → solution → concept test → cumulative mock. Do not jump straight to answer keys.

What this programme is—and is not

It teaches transferable foundations used across product companies. It is not a promise that every company uses the same interview rounds, and it is not a list of leaked questions. Job descriptions differ: an analyst role may emphasise SQL, product metrics, statistics and BI; an engineering role may emphasise SQL, Python, DBMS, pipelines and cloud. Some companies also test DSA, aptitude, take-home work or tool-specific skills.

Learning rules

1. **Attempt first.** Keep solutions in a separate file or print the exercise pages without answer sections.

2. **Write, don't just read.** Type every SQL query and Python solution. Change sample data and rerun.

3. **Explain aloud.** For every solution, explain the grain, assumptions, edge cases and complexity.

4. **Maintain an error log.** Record the error, why it happened, the corrected rule and a new example.

5. **Use spaced repetition.** Reattempt missed questions after 1 day, 3 days and 7 days.

6. **No memorised claims.** For projects and resumes, explain only what was actually implemented.

7. **Use PostgreSQL for SQL practice.** Syntax differs across PostgreSQL, MySQL, SQL Server, Oracle and SQLite. Learn the concept, then check the target dialect.

Recommended 20-week route

| Weeks | Study block | Required output |

|---|---|---|

| 1 | Relational data, tables, keys, constraints and SQL setup | Create a small schema and load sample rows |

| 2–3 | SQL querying, joins, aggregation and NULLs | 35 beginner queries with verified results |

| 4 | Subqueries, CTEs, set operations and conditional logic | 15 intermediate queries |

| 5–6 | Window functions, dates, deduplication and advanced SQL | 20 interview queries under time |

| 7 | DBMS internals, normalisation, transactions and ACID | Explain and demonstrate transactions |

| 8 | Isolation, locks, indexes and query plans | Investigate 3 deliberately slow queries |

| 9–10 | Python fundamentals and problem solving | 25 small programs without pandas |

| 11 | Files, exceptions, modules, OOP and testing | Build a validated CSV processing script |

| 12 | pandas and data cleaning | Reproduce an analysis from raw CSV |

| 13 | Statistics and experimentation | Explain a product experiment and uncertainty |

| 14 | Excel, Power Query and Power BI | Build and reconcile a small dashboard |

| 15 | Data modelling and warehouse fundamentals | Design a star schema and explain its grain |

| 16–17 | Data engineering pipelines, APIs, reliability and cloud concepts | Build a rerunnable batch pipeline |

| 18 | Project deep-dives and resume defence | Explain each project in 5 minutes |

| 19 | Analyst and engineering role-specific practice | Complete one role-specific mock each |

| 20 | Full interview simulations and revision | Complete two timed mock loops |

A student with less time can compress the calendar, but should not skip SQL, DBMS, Python, project explanation or timed practice.

Daily routine (2–3 hours)

• 15 min: recall yesterday's material without notes.

• 35 min: learn one new concept.

• 45 min: solve guided exercises.

• 35 min: solve independent exercises.

• 20 min: explain the concept aloud and write an error-log entry.

• 10 min: review a previously missed question.

How to use the solutions

A solution is not just code. Compare:

• Does it answer the exact question?

• Does it preserve the intended row grain?

• What happens with NULL, duplicates, ties, empty input or invalid values?

• Is there a simpler or more scalable approach?

• Can you explain why it works?

Readiness gates

SQL: solve joins, aggregation, subqueries, windows, date logic and deduplication accurately under time.

DBMS: explain keys, normalisation, ACID, isolation, deadlocks, indexes and execution plans with examples.

Python: solve list/dictionary/set problems, validate input, test edge cases and explain complexity.

Analytics: define metrics, interpret distributions and uncertainty, diagnose product changes.

BI: model data, write measures, reconcile totals and design a decision-oriented dashboard.

Data engineering: design an incremental, idempotent, observable pipeline with failure recovery.

Projects: trace data from source to output and explain limitations honestly.

Tool setup

Recommended local setup:

• PostgreSQL + pgAdmin or DBeaver for SQL/DBMS exercises.

• Python 3.11+ and VS Code.

• Jupyter Notebook for pandas/statistics exercises.

• Git and GitHub for version control.

• Power BI Desktop for BI exercises (Windows; on macOS use a suitable alternative such as Power BI Service where available, or Tableau Public/Looker Studio for visualisation practice).

Use the smallest tools needed. Do not spend weeks configuring infrastructure instead of learning the fundamentals.

External practice — how to use the links in each module

Use the external platforms listed at the end of each module, not a random playlist of sites. The internal exercises teach the concept; external platforms provide independent problems and automated feedback. Keep a tracker with columns: concept, platform, problem URL/title, first attempt date, result, mistake, retry date.

Rules: do not copy solutions on the first attempt; solve at least one problem without a timer, then repeat under a timer; write down the dialect/language used; re-solve every failed problem after 1, 3 and 7 days. Free access and platform content can change; check the platform before planning a paid subscription.